

Universitatea din București
Facultatea de Matematică și Informatică
Specializarea Informatică

Machine Learning

Detectia și numărarea obiectelor în semnale radio
zgomotoase

Student: Sava Darius-Ștefan

Grupa: 242

Disciplina: Învățare Automată (Machine Learning)

Cuprins

1	Introducere și explorarea datelor	1
2	Primele încercări: K-NN și SVM	1
2.1	Matricele de confuzie pentru modelele clasice	2
3	Procesarea imaginilor	2
3.1	Citirea imaginilor în grayscale	3
3.2	Augmentarea imaginilor	3
3.3	Normalizarea imaginilor	4
4	Blocul rezidual	4
4.1	Ideea principală a shortcut-ului	5
4.2	Alinierea dimensiunilor în shortcut	5
5	Rețeaua CNN propusă	5
5.1	Privire de ansamblu asupra arhitecturii	5
5.2	Parametrii importanți ai convoluțiilor	6
5.3	Creșterea numărului de canale	6
5.4	MaxPooling și pooling asimetric	7
5.5	AdaptiveAvgPool și clasificatorul final	7
6	Antrenarea modelului	8
6.1	Funcția de pierdere: CrossEntropyLoss	8
6.2	Optimizatorul Adam și scheduler-ul	8
6.3	Salvarea celui mai bun model	8
7	Comitetul de experți	9
8	Test-Time Augmentation și votul final	10
9	Rezultate experimentale	10
9.1	Comparația variantelor testate	10
9.2	Comparația celor 5 experți	12
10	Concluzii	13
11	Materialele din care m-am documentat	14

1. Introducere și explorarea datelor

Când am citit pentru prima dată cerința proiectului – ”Participants are asked to employ machine learning methods to detect (count) objects in noisy radio signals” – mi s-a părut o temă extrem de interesantă, dar și provocatoare. Primul pas pe care l-am făcut a fost să explorez vizual imaginile din setul de antrenare pentru a înțelege exact cu ce fel de date lucrez și în ce constă, de fapt, proiectul.

Analizând spectrogramele, am observat că într-un semnal radio, adică într-o singură imagine, pot exista 1, 2, 3, 4 sau 5 obiecte. Această observație m-a condus la prima decizie importantă: am abordat problema ca pe o sarcină de clasificare, unde fiecare număr de obiecte reprezintă o clasă distinctă. Această alegere este mai potrivită decât o regresie, deoarece ieșirea finală trebuie să fie una dintre cele cinci clase posibile.

Datele nu arată ca imagini clasice, în care obiectul este ușor de separat vizual. Spectrogramele conțin urme, intensități și mult zgomot de fundal. Din acest motiv, modelul trebuie să învețe tipare vizuale din semnal, nu doar să compare valori brute de pixeli.

2. Primele încercări: K-NN și SVM

Pentru a avea un punct de plecare, am testat mai întâi două modele clasice de Machine Learning: *K-Nearest Neighbors* și *Support Vector Machine*. Scopul lor nu a fost să ofere soluția finală, ci să funcționeze ca baseline-uri cu care să pot compara apoi performanța rețelelor convoluționale.

Spre deosebire de un CNN, aceste modele nu lucrează natural direct pe structură spațială, ci au nevoie de o reprezentare numerică fixă a fiecărei imagini. Din acest motiv, spectrogramele au fost tratate într-o formă simplificată, potrivită pentru clasificatori clasici, fără filtre învățate automat și fără mecanisme dedicate pentru robustețe la mici deplasări sau la zgomot. Această alegere a fost utilă pentru comparație, deoarece a evidențiat foarte clar diferența dintre un model care primește o reprezentare statică și unul care își învață singur trăsăturile relevante.

Model	Acuratețe obținută
K-NN	21.392%
SVM	27.145%

Se observă că SVM a depășit K-NN, dar ambele rezultate au rămas mult sub performanța obținută ulterior de modelele CNN. Acest lucru sugerează că, pentru această problemă, informația utilă nu este ușor de separat printr-o reprezentare fixă și simplă a imaginii. Spectrogramele conțin tipare locale și variații spațiale fine, iar aceste modele clasice nu reușesc să exploateze suficient de bine structura lor.

2.1. Matricele de confuzie pentru modelele clasice

Pentru a înțelege mai clar de ce modelele clasice nu au fost suficiente pentru această problemă, am analizat și matricile de confuzie corespunzătoare pentru K-NN și SVM. Aceste figuri arată mai bine unde apar erorile și confirmă faptul că modelele clasice au dificultăți în a separa corect clasele apropiate.

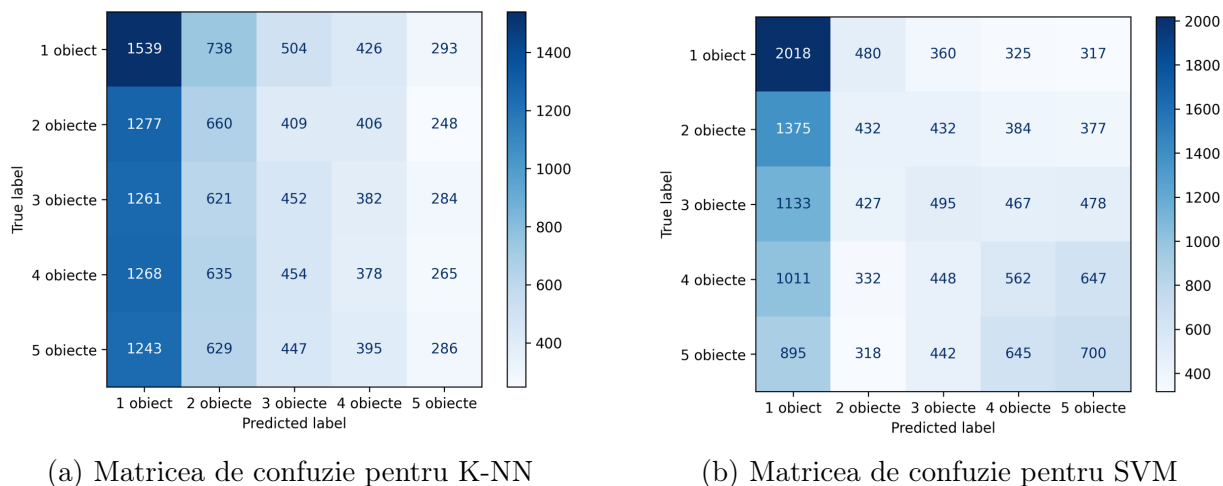


Figura 1: Matricele de confuzie pentru modelele clasice testate.

Aceste rezultate întăresc concluzia că baseline-urile clasice pot oferi un punct de plecare, dar nu reușesc să surprindă suficient de bine structurile vizuale din spectrograme, mai ales în prezența zgomotului.

Așteptările mele erau relativ optimiste, însă rezultatele au fost slabe. Intenția mea inițială era să trimit aceste rezultate pentru a avea o primă submisie. Totuși, când am intrat pe platformă și am văzut că mulți dintre colegii mei obținuseră deja scoruri de peste 65% pe leaderboard, mi-a fost, sincer, rușine să trimit o soluție cu o performanță atât de scăzută.

Acela a fost momentul în care am realizat limitările metodelor clasice. K-NN și SVM nu extrag singure trăsături vizuale complexe din imagine. Ele primesc imaginea într-o formă mai brută și nu tratează natural faptul că aceeași structură poate apărea în poziții ușor diferite. În plus, zgomotul de fundal le afectează puternic.

De aici am trecut la Deep Learning, mai exact la rețele neuronale convoluționale. CNN-urile sunt potrivite pentru această problemă deoarece învață direct filtre peste imagine și pot extrage singure forme locale, apoi tipare tot mai complexe. După mai mult research, am început să construiesc primul model CNN de la zero.

3. Procesarea imaginilor

Partea de procesare a imaginilor este esențială deoarece modelul nu primește direct fișierele din folder, ci tensori pregătiți pentru antrenare. În cod, această etapă este realizată

prin clasa `CustomImageDataset`. Rolul ei este să citească imaginea, să aplice augmentarea când este cazul, să normalizeze valorile și să returneze imaginea împreună cu eticheta.

```
1 class CustomImageDataset(Dataset):
2     def __init__(self, tabel_date, folder_imagini, e_test=False, augmentare=False):
3         self.tabel_date=tabel_date
4         self.folder_imagini=folder_imagini
5         self.e_test=e_test
6         self.augmentare=augmentare
```

3.1. Citirea imaginilor în grayscale

Spectrogramele sunt citite în format alb-negru, deoarece informația importantă este dată de intensitatea semnalului și de forma urmelor din imagine, nu de culori RGB. Din acest motiv, primul strat al rețelei primește un singur canal.

```
1 poza_citita=read_image(cale_poza, mode=ImageReadMode.GRAY)
2 poza=poza_citita.type(torch.float32)
```

Conversia la `float32` este necesară deoarece rețeaua lucrează cu valori reale și calculează operații precum convoluții, normalizări, gradient și actualizări de ponderi. Pixelii nu mai sunt tratați ca valori întregi brute, ci ca valori numerice pe care modelul le poate transforma în trăsături.

3.2. Augmentarea imaginilor

Augmentarea are rolul de a face modelul mai robust. Dacă rețeaua vede mereu exact aceeași imagine, există riscul să memoreze poziții și detalii foarte specifice. În schimb, dacă imaginea este modificată ușor la fiecare epocă, modelul este forțat să învețe forma generală a semnalului.

```
1 if self.augmentare:
2     mutare=random.randint(-6, 6)
3     if mutare!=0:
4         poza=torch.roll(poza, shifts=mutare, dims=2)
5
6     zgomot=torch.randn_like(poza)
7     zgomot_scalat=zgomot * 0.03
8     poza=poza + zgomot_scalat
```

Prima augmentare este deplasarea orizontală cu `torch.roll`. Rolul acestei operații este să arate modelului că obiectele pot apărea puțin mai la stânga sau mai la dreapta, fără ca eticheta să se schimbe. Astfel, modelul nu învață doar poziția exactă a unei urme, ci structura ei.

A doua augmentare este zgomotul Gaussian de intensitate mică, 0.03. Această operație simulează mici variații ale imaginii și reduce dependența de detalii fragile. Practic, modelul este obligat să se concentreze pe informația stabilă, nu pe "puricii" de fundal.

3.3. Normalizarea imaginilor

După citire și augmentare, imaginea este normalizată folosind media și deviația standard proprii.

```
1 medie=poza.mean()
2 deviatie=poza.std()
3
4 if deviatie > 0:
5     diferenta=poza - medie
6     poza=diferenta / deviatie
7 else:
8     poza=poza - medie
```

Rolul normalizării este să stabilizeze valorile care intră în rețea. Dacă imaginile au scări numerice foarte diferite, antrenarea devine mai instabilă. Prin normalizare, fiecare imagine este adusă într-o zonă numerică mai controlată, iar modelul poate învăța mai ușor filtre utile.

4. Blocul rezidual

Blocul rezidual este una dintre componentele cele mai importante ale modelului. Într-o rețea simplă, informația trece prin mai multe straturi de convoluție, iar fiecare strat modifică rezultatul anterior. Când rețeaua devine mai adâncă, există riscul ca informația utilă extrasă la început să fie slăbită sau modificată prea mult.

Blocul rezidual rezolvă această problemă printr-o conexiune directă, numită shortcut. Pe lângă drumul normal prin convoluții, intrarea blocului este trimisă și direct spre final, unde se adună cu rezultatul obținut după filtre.

```
1 class BlocRezidual(nn.Module):
2     def __init__(self, in_channels, out_channels, pas=1):
3         super().__init__()
4         self.conv1=nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=pas,
5         ↪ padding=1, bias=False)
6         self.bn1=nn.BatchNorm2d(out_channels)
7         self.conv2=nn.Conv2d(out_channels, out_channels, kernel_size=3, stride=1,
8         ↪ padding=1, bias=False)
9         self.bn2=nn.BatchNorm2d(out_channels)
```

4.1. Ideea principală a shortcut-ului

În forward, se vede exact mecanismul blocului rezidual:

```
1 iesire_scurtatura=self.scurtatura(x)
2 rezultat=rezultat + iesire_scurtatura
```

Rolul shortcut-ului este să păstreze o copie a informației inițiale. Astfel, blocul nu trebuie să reconstruiască totul de la zero, ci învață ce trebuie adăugat sau corectat peste informația deja existentă. Formula poate fi scrisă simplu astfel:

$$y = F(x) + x \quad (1)$$

unde $F(x)$ este transformarea realizată de convoluții, iar x este informația transmisă prin shortcut. Această idee ajută rețeaua să fie mai stabilă și să nu piardă trăsăturile utile pe măsură ce devine mai adâncă.

4.2. Alinierea dimensiunilor în shortcut

Uneori, numărul de canale al intrării nu este egal cu numărul de canale al ieșirii. De exemplu, un bloc poate primi 64 de canale și poate produce 128. În acest caz, intrarea nu mai poate fi adăugată direct peste ieșire, deoarece tensorii nu au aceeași formă.

Pentru această situație se folosește o convoluție de tip 1×1 pe shortcut:

```
1 if pas != 1 or in_channels != out_channels:
2     strat_conv_scurtatura=nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=
    ↪ pas, bias=False)
3     strat_bn_scurtatura=nn.BatchNorm2d(out_channels)
```

Rolul acestei convoluții nu este să caute forme complicate, ci să potrivească dimensiunile. Ea transformă intrarea astfel încât adunarea reziduală să fie posibilă. Practic, shortcut-ul rămâne ideea principală, iar convoluția 1×1 doar îl face compatibil cu restul blocului.

5. Rețeaua CNN propusă

După definirea blocului rezidual, am construit rețeaua completă. Modelul pornește de la o spectrogramă grayscale, extrage treptat trăsături prin convoluții și blocuri reziduale, comprimă reprezentarea prin pooling, iar la final produce 5 scoruri, câte unul pentru fiecare clasă.

5.1. Privire de ansamblu asupra arhitecturii

Privită per total, rețeaua mea este alcătuită dintr-un strat convoluțional inițial, 4 blocuri reziduale, 3 straturi de MaxPooling, un strat de AdaptiveAvgPool2d și un clasificator

final format din două straturi **Linear**. Mai concret, fluxul principal al datelor este următorul:

$$1 \text{ canal} \rightarrow 64 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 128 \rightarrow 5 \text{ clase}$$

La nivel de clasificare finală, rețeaua folosește un strat liniar de la 512 la 128 de neuroni, urmat de activare **LeakyReLU**, **Dropout(0.4)** și încă un strat liniar de la 128 la 5 neuroni, corespunzător celor 5 clase posibile. Dacă mă uit strict la straturile care învață efectiv parametri, arhitectura conține un strat convoluțional inițial, câte două convoluții în fiecare bloc rezidual, trei shortcut-uri de tip 1×1 în blocurile unde se schimbă numărul de canale, iar la final două straturi liniare. Cu alte cuvinte, modelul crește treptat complexitatea reprezentării, iar decizia finală este luată abia după ce imaginea a fost trecută prin mai multe niveluri de filtrare și comprimare.

5.2. Parametrii importanți ai convoluțiilor

Primul strat convoluțional este:

```
1 self.strat_initial=nn.Sequential(  
2     nn.Conv2d(1, 64, kernel_size=(3, 7), padding=(1, 3), bias=False),  
3     nn.BatchNorm2d(64),  
4     nn.LeakyReLU(0.1)  
5 )
```

În acest strat, `in_channels=1` apare deoarece imaginea este grayscale și are un singur canal. `out_channels=64` înseamnă că stratul produce 64 de hărți de trăsături. Fiecare hartă este rezultatul unui filtru diferit, iar rolul acestor filtre este să identifice structuri locale diferite din spectrogramă.

`kernel_size=(3,7)` stabilește dimensiunea ferestrei cu care filtrul analizează imaginea. Kernel-ul este mai lat pe orizontală deoarece multe urme din spectrograme au structură întinsă pe această direcție. Această alegere oferă filtrului mai mult context orizontal încă din primul strat.

`padding=(1,3)` adaugă margini artificiale în jurul imaginii, astfel încât dimensiunea să nu se micșoreze prea repede după convoluție. Rolul padding-ului este să protejeze și informația din apropierea marginilor.

`stride` controlează pasul cu care filtrul se deplasează peste imagine. În modelul meu, convoluțiile folosesc în general pas mic, deoarece diferențele dintre clase pot fi subtile și nu vreau ca rețeaua să sară peste detalii importante.

5.3. Creșterea numărului de canale

Rețeaua folosește patru blocuri reziduale, iar numărul de canale crește progresiv:


```
1 self.bloc1=BlocRezidual(64, 64)
2 self.bloc2=BlocRezidual(64, 128)
3 self.bloc3=BlocRezidual(128, 256)
4 self.bloc4=BlocRezidual(256, 512)
```

Rolul acestei creșteri este să mărească puterea de reprezentare a modelului. Primele straturi extrag forme mai simple, iar straturile mai adânci combină aceste forme în tipare mai complexe. De exemplu, diferența dintre 3, 4 și 5 obiecte poate fi foarte fină, iar modelul are nevoie de suficiente canale pentru a descrie aceste diferențe.

5.4. MaxPooling și pooling asimetric

Între blocurile reziduale apar straturi de MaxPool2d:

```
1 self.pool1=nn.MaxPool2d(kernel_size=(1, 2),stride=(1, 2))
2 self.pool2=nn.MaxPool2d(kernel_size=(2, 2),stride=(2, 2))
3 self.pool3=nn.MaxPool2d(kernel_size=(1, 2),stride=(1, 2))
```

MaxPool2d reduce dimensiunea reprezentării, păstrând valorile cele mai puternice din ferestre locale. Rolul lui este să comprime imaginea, să reducă zgomotul și să scadă costul de calcul, fără să piardă imediat informația dominantă.

Am folosit și pooling asimetric, de tip (1, 2). Această alegere comprimă mai ales lățimea imaginii și păstrează mai bine dimensiunea verticală. Motivul este că spectrogramele au o înălțime relativ mică, iar o reducere prea agresivă pe verticală ar putea elimina detalii utile. Prin pooling asimetric, rețeaua reduce treptat reprezentarea, dar păstrează mai multă informație pe înălțime.

5.5. AdaptiveAvgPool și clasificatorul final

La finalul părții convoluționale, modelul folosește AdaptiveAvgPool2d((1,1)):

```
1 self.gap=nn.AdaptiveAvgPool2d((1, 1))
```

Rolul acestui strat este să transforme fiecare hartă de trăsături într-o singură valoare medie. Dacă modelul are 512 canale înainte de acest pas, după AdaptiveAvgPool rămân 512 valori. Aceste valori rezumă informația extrasă de fiecare canal.

Apoi urmează clasificatorul final:

```
1 self.decizie_finala=nn.Sequential(
2     nn.Flatten(),
3     nn.Linear(512, 128),
4     nn.LeakyReLU(0.1),
5     nn.Dropout(0.4),
6     nn.Linear(128, 5)
7 )
```

Flatten transformă tensorul într-un vector. **Linear(512,128)** combină cele 512 trăsături într-o reprezentare mai compactă, iar **Linear(128,5)** produce cele 5 scoruri finale. **Dropout(0.4)** are rol de regularizare: în timpul antrenării, oprește aleator o parte din activări, astfel încât modelul să nu devină dependent de câțiva neuroni. Această operație reduce riscul de overfitting.

6. Antrenarea modelului

Antrenarea modelului nu se rezumă doar la rularea unei bucle de epoci. Componentele importante sunt funcția de pierdere, optimizatorul, scheduler-ul și modul în care este salvat cel mai bun model.

6.1. Funcția de pierdere: **CrossEntropyLoss**

Pentru că problema este una de clasificare cu 5 clase, funcția de pierdere potrivită este **CrossEntropyLoss**. Aceasta compară scorurile produse de rețea cu eticheta corectă și penalizează predicțiile greșite.

```
1 loss_funciton=nn.CrossEntropyLoss(label_smoothing=0.1)
```

Am folosit `label_smoothing=0.1`, deoarece modelul nu trebuie să devină exagerat de sigur pe o singură clasă. Această tehnică face țintele puțin mai flexibile și ajută la generalizare. În practică, reduce tendința rețelei de a da predicții prea "arogante".

6.2. Optimizatorul Adam și scheduler-ul

Optimizatorul folosit este Adam:

```
1 optimizator=torch.optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-4)
```

Rolul optimizatorului este să actualizeze ponderile rețelei după fiecare batch. Valoarea `lr=0.001` a fost stabilă în experimente, iar `weight_decay=1e-4` funcționează ca regularizare, penalizând ponderile prea mari.

Pentru rata de învățare am folosit **CosineAnnealingLR**:

```
1 scheduler=torch.optim.lr_scheduler.CosineAnnealingLR(optimizator, T_max=NUM_EOCHS)
```

Rolul scheduler-ului este să reducă treptat learning rate-ul pe parcursul antrenării. La început, modelul poate face pași mai mari, iar spre final pașii devin mai mici, ceea ce ajută la o convergență mai stabilă.

6.3. Salvarea celui mai bun model

În timpul antrenării, nu am păstrat neapărat ultima epocă, ci modelul cu cea mai bună acuratețe pe validare:

```
1 if acuratete > cea_mai_buna_acuratete:
2     cea_mai_buna_acuratete=acuratete
3     stare_model=model.state_dict()
4     torch.save(stare_model, nume_fisier_model)
```

Această alegere este importantă deoarece ultima epocă nu este obligatoriu cea mai bună. Uneori modelul poate începe să se adapteze prea mult pe train, iar performanța pe validare să nu mai crească. Salvarea celui mai bun model pe validare păstrează varianta cea mai utilă pentru predicții noi.

7. Comitetul de experți

După ce am obținut rezultate bune cu un singur CNN, am vrut să cresc stabilitatea soluției. Ideea de bază a fost următoarea: o problemă dificilă poate fi rezolvată mai sigur de mai mulți oameni decât de unul singur. Un singur om poate greși o situație, dar ceilalți pot observa greșeala și pot ajunge la răspunsul corect. Am aplicat aceeași idee și pentru modele.

În loc să folosesc o singură rețea, am antrenat 5 rețele independente, numite în cod experți. Fiecare expert vede datele într-o împărțire diferită, iar la final toate modelele participă la decizia finală.

```
1 NUM_MODELE=5
2 impartitor_date=StratifiedKFold(n_splits=NUM_MODELE, shuffle=True, random_state=42)
```

StratifiedKFold împarte datele în 5 folduri, păstrând distribuția claselor. Fiecare model este antrenat pe 4 folduri și validat pe foldul rămas. Prin această metodă, fiecare expert învață dintr-o perspectivă ușor diferită asupra datelor.

După antrenare, toți experții sunt încărcăți într-o listă:

```
1 comitet_experti=[]
2 for i in range(NUM_MODELE):
3     model_incarcat=NeuralNetwork().to(device)
4     cale_salvare=f'model_expert_{i}.pth'
5     model_incarcat.load_state_dict(torch.load(cale_salvare))
6     model_incarcat.eval()
7     comitet_experti.append(model_incarcat)
```

Rolul comitetului este să reducă efectul erorilor individuale. Dacă un expert greșește o spectrogramă, ceilalți experți pot compensa prin voturile lor. Rezultatul final devine astfel mai stabil decât predicția unui singur model.

8. Test-Time Augmentation și votul final

La inferență, fiecare expert nu vede doar imaginea originală, ci și două versiuni ușor deplasate orizontal. Această tehnică se numește Test-Time Augmentation.

```
1 for expert in comitet_experti:
2     vot_normal=expert(image_batch)
3
4     imagine_mutata_stanga=torch.roll(image_batch, shifts=-5, dims=3)
5     vot_mutat_stanga=expert(imagine_mutata_stanga)
6
7     imagine_mutata_dreapta=torch.roll(image_batch, shifts=5, dims=3)
8     vot_mutat_dreapta=expert(imagine_mutata_dreapta)
```

Rolul TTA este să verifice dacă predicția rămâne stabilă atunci când imaginea este ușor mutată. Dacă modelul înțelege forma semnalului, nu ar trebui să își schimbe radical decizia pentru o deplasare mică.

Voturile sunt combinate ponderat:

```
1 vot_normal_ponderat=vot_normal * 0.5
2 vot_stanga_ponderat=vot_mutat_stanga * 0.25
3 vot_dreapta_ponderat=vot_mutat_dreapta * 0.25
4
5 vot_total_expert=(vot_normal_ponderat + vot_stanga_ponderat + vot_dreapta_ponderat)
6 suma_voturi=suma_voturi + vot_total_expert
```

Imaginea originală primește ponderea cea mai mare, deoarece este varianta principală. Cele două imagini deplasate au rol de stabilizare. La final, voturile celor 5 experți sunt mediate:

```
1 vot_comitet=suma_voturi / NUM_MODELE
```

În total, fiecare imagine de test este analizată de 15 ori: 5 experți, fiecare cu 3 versiuni ale imaginii. Această combinație reduce variația predicției și a fost una dintre componentele decisive pentru scorul final.

9. Rezultate experimentale

Evoluția rezultatelor a fost treptată. Modelele clasice au avut performanțe slabe, iar îmbunătățirile reale au apărut odată cu trecerea la rețele convoluționale reziduale, introducerea *Test-Time Augmentation* și, în final, folosirea unui ansamblu de experți.

9.1. Compararea variantelor testate

Pentru a urmări mai clar evoluția soluției, am comparat principalele variante CNN testate. În același timp, acest tabel rezumă și procesul de reglare a configurației mode-

lului, prin modificarea treptată a unor componente precum optimizatorul, scheduler-ul, regularizarea, strategia de inferență și numărul de modele combinate.

Tabela 1: Comparația principalelor variante CNN testate

Scor (%)	Variantă	Modificări / hiperparametri
73.745	CNN rezidual inițial	SiLU, AdamW, Dropout 0.5, weight decay mai mare, 60 epoci.
75.054	CNN rezidual îmbunătățit	LeakyReLU, Adam, Dropout 0.4, weight decay 10^{-4} , ReduceLROnPlateau, 40 epoci.
75.563	CNN + TTA	Configurația anterioară + Test-Time Augmentation la inferență.
75.927	CNN optimizat	Variantă mai robustă, cu antrenare și inferență mai bine corelate.
77.672	3 modele + TTA	3 modele antrenate separat, combinate prin vot mediu, împreună cu TTA.
78.836	Model final: 5 experți + TTA	Stratified 5-Fold, Adam, label smoothing 0.1, CosineAnnealingLR, batch size 64, TTA ponderat.

Tabelul arată că îmbunătățirea performanței nu a venit dintr-o singură schimbare, ci din reglarea treptată a mai multor componente. Am testat variante diferite de activare, optimizator, regularizare, scheduler, număr de epoci și strategie de inferență, iar configurația finală a fost aleasă pe baza scorului obținut.

9.2. Comparația celor 5 experți

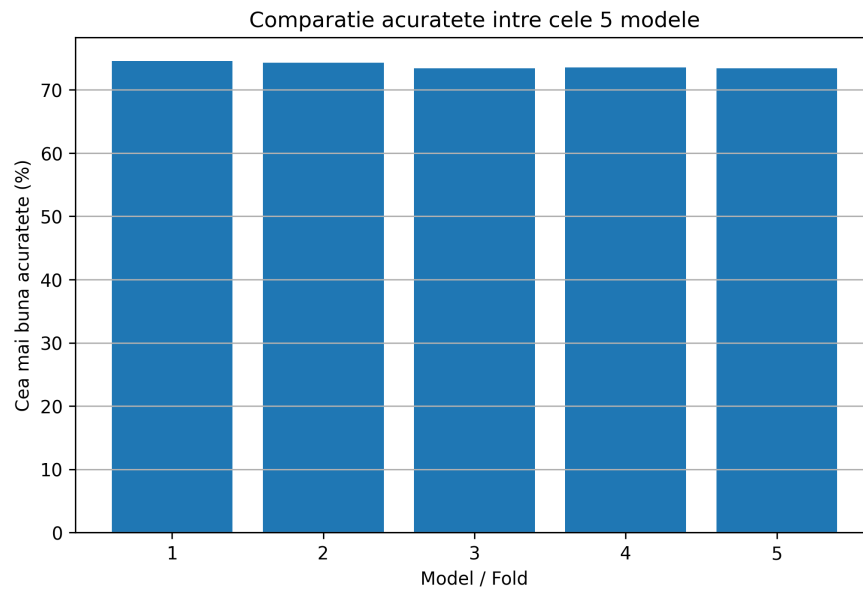
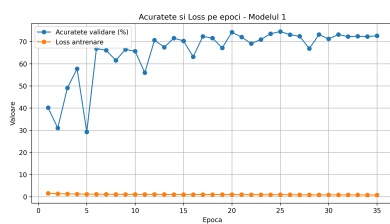
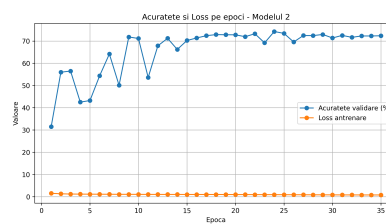


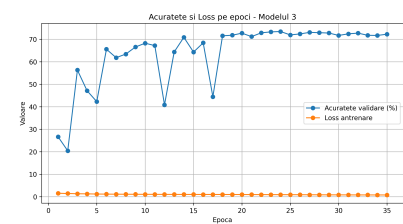
Figura 2: Comparația acurateții obținute de cei 5 experți antrenați prin Stratified 5-Fold Cross-Validation.



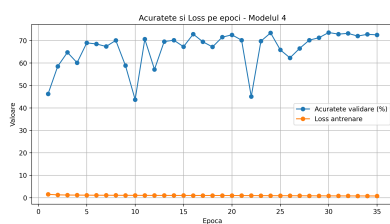
(a) Expertul 1



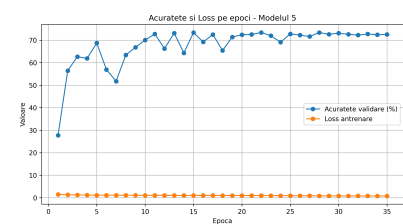
(b) Expertul 2



(c) Expertul 3



(d) Expertul 4



(e) Expertul 5

Figura 3: Evoluția acurateții și a funcției de pierdere pentru cei 5 experți.

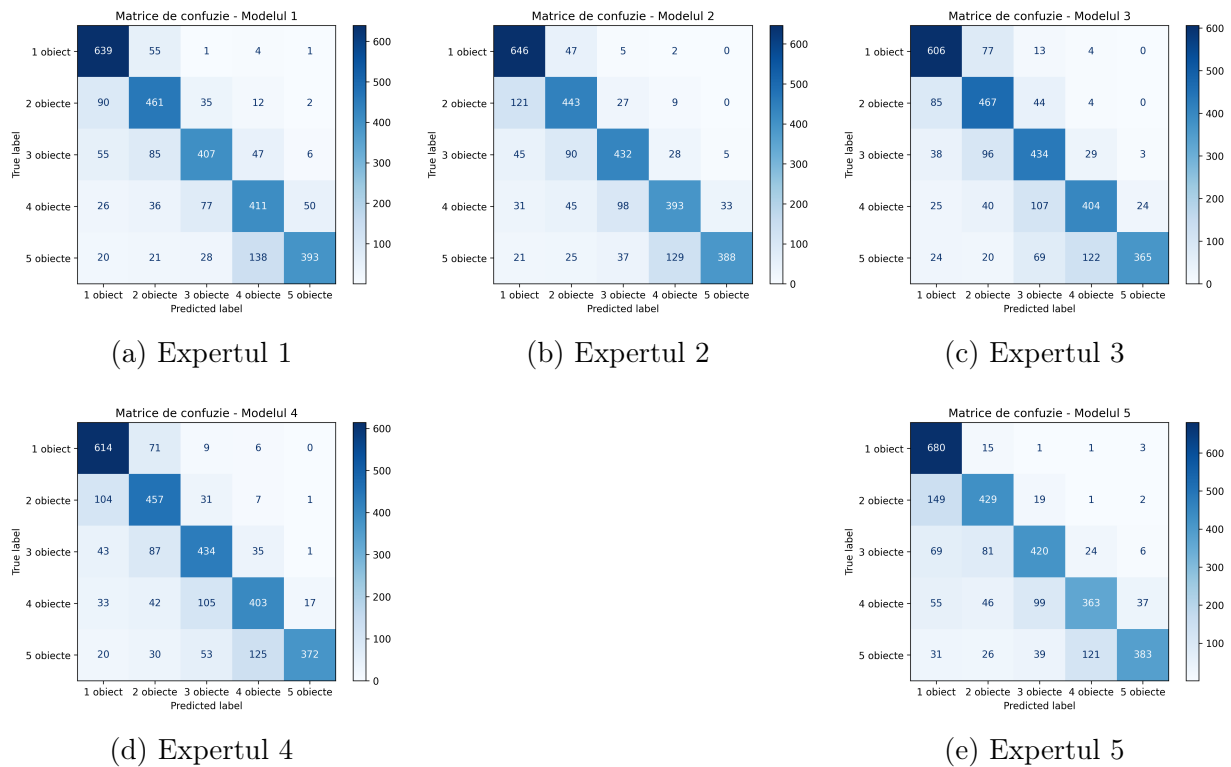


Figura 4: Matricele de confuzie pentru experții antrenati pe folduri diferite.

Graficele arată că experții au performanțe apropiate, ceea ce susține ideea de comitet. Matricele de confuzie confirmă faptul că cele mai multe predicții corecte sunt pe diagonală, iar erorile apar mai ales între clase apropiate numeric.

10. Concluzii

Proiectul a pornit de la o cerință aparent simplă, dar datele au arătat rapid că problema este dificilă. Modelele clasice, K-NN și SVM, nu au reușit să extragă suficientă informație din spectrograme. Trecerea la CNN a fost pasul decisiv, deoarece rețeaua convoluțională poate învăța singură filtre potrivite pentru formele din imagini.

Cele mai importante componente ale soluției finale au fost procesarea imaginilor, augmentarea, blocul rezidual, arhitectura CNN, pooling-ul asimetric, funcția de pierdere cu label smoothing, comitetul de 5 experți și Test-Time Augmentation. Fiecare componentă a avut un rol clar, iar combinate au dus la o soluție mult mai stabilă decât un singur model antrenat simplu.

Scorul final a fost de 78.836% pe leaderboard-ul public, unde soluția s-a clasat pe locul 31, respectiv 77.115% pe leaderboard-ul private, unde s-a clasat pe locul 49. Aceste rezultate au fost obținute prin combinația dintre arhitectură, regularizare și vot între mai multe modele. Cel mai important lucru din proiect a fost că performanța nu a venit dintr-o singură modificare, ci din mai multe decizii care s-au completat între ele.

11. Materialele din care m-am documentat

Pe parcursul realizării proiectului, m-am documentat din mai multe materiale introductive și practice despre rețele neuronale convoluționale, PyTorch și clasificarea imaginilor. Mai jos sunt principalele surse pe care le-am folosit pentru a înțelege conceptele de bază și pentru a-mi clarifica anumite idei legate de implementare:

- <https://nafizshahriar.medium.com/what-is-convolutional-neural-network-cnn-deep-learning-b3921bdd82d5>
- <https://madhuramiah.medium.com/convolution-neural-network-the-backbone-of-image-classification-ddd2a49a6efa>
- <https://www.datacamp.com/tutorial/pytorch-cnn-tutorial>
- <https://codesignal.com/learn/courses/introduction-to-drawing-recognition-and-cnn-fundamentals/lessons/cnn-fundamentals>
- <https://medium.com/machine-learning-researcher/convlutional-neural-network-cnn-2fc4faa7bb63>
- https://www.youtube.com/watch?v=wnK3uWv_WkU
- <https://www.digitalocean.com/community/tutorials/writing-cnns-from-scratch-in-pytorch>
- <https://www.youtube.com/watch?v=jL2G8DG-qmI>
- <https://www.youtube.com/watch?v=pj9-rr1wDhM>